Trident 연산자를 사용하여 구축

Astra Trident

NetApp
March 21, 2022

This PDF was generated from https://docs.netapp.com/ko-kr/trident/trident-get-started/kubernetes-customize-deploy.html on March 21, 2022. Always check docs.netapp.com for the latest.

목차

Trident 연산자를 사용하여 구축 1
Hrom을 사용하여 Trident 연산자를 배포합니다 1
Trident 연산자를 수동으로 배포합니다 2
Trident 운영자 배포를 사용자 지정합니다 7

Trident 연산자를 사용하여 구축

Trident 운영자와 함께 Astra Trident를 배포할 수 있습니다. Trident 연산자는 수동으로 또는 Hrom을 사용하여 배포할 수 있습니다.

에 아직 익숙하지 않은 경우 "기본 개념"이제 아주 좋은 시간입니다.

Astra Trident를 구축하려면 다음과 같은 사전 요구 사항을 충족해야 합니다.

- Kubernetes 1.17 이상을 실행하는 지원되는 Kubernetes 클러스터에 대한 모든 권한이 있습니다.
- 지원되는 NetApp 스토리지 시스템에 액세스할 수 있습니다.
- 모든 Kubernetes 작업자 노드에서 볼륨을 마운트할 수 있습니다.
- 사용할 Kubernetes 클러스터를 관리하도록 "kubbtl"(또는 OpenShift를 사용하는 경우 OC)가 설치 및 구성된 Linux 호스트를 보유하고 있습니다.
- Kubernetes 클러스터 구성을 가리키도록 'KUBECONFIG' 환경 변수를 설정했습니다.
- 을(를) 활성화했습니다 "Astra Trident에서 요구하는 기능 게이트".
- Docker Enterprise와 함께 Kubernetes를 사용하는 경우, "다음 단계에 따라 CLI 액세스를 설정합니다".

다 잡았나요? 좋습니다! 그럼 이제 시작하겠습니다.

Hrom을 사용하여 Trident 연산자를 배포합니다

나열된 단계를 수행하여 Helm을 사용하여 Trident 연산자를 배포합니다.

위에 나열된 필수 구성 요소 외에도 Hrom을 사용하여 Trident 연산자를 구축하려면 다음이 필요합니다.

- Kubernetes 1.17 이상
- Helm 버전 3

단계

1. Trident의 제어 리포지토리 추가:

```
helm repo add netapp-trident https://netapp.github.io/trident-helm-chart
```

2. 'helm install' 명령을 사용하여 배포 이름을 지정합니다. 다음 예를 참조하십시오.

```
helm install <release-name> netapp-trident/trident-operator --version
22.1.0 --namespace <trident-namespace>
```

Trident의 네임스페이스를 아직 만들지 않은 경우 '--create-namespace' 매개 변수를 'helm install' 명령에 추가할 수 있습니다. 그런 다음 Helm이 자동으로 네임스페이스를 만듭니다.

설치 중에 구성 데이터를 전달하는 방법에는 두 가지가 있습니다.

- '--Values'(또는 '-f'): 재정의가 있는 YAML 파일을 지정합니다. 이 옵션은 여러 번 지정할 수 있으며 가장 오른쪽 파일이 우선 적용됩니다.
- '--set': 명령행에서 overrides를 지정합니다.

예를 들어, 기본값인 debug를 변경하려면 다음 '--set' 명령을 실행합니다.

```
$ helm install <name> netapp-trident/trident-operator --version 22.1.0
--set tridentDebug=true
```

제어 차트의 일부인 Values.YAML 파일은 키 목록과 기본값을 제공합니다.

'Helm list'는 이름, 네임스페이스, 차트, 상태, 앱 버전, 개정 번호 등

Trident 연산자를 수동으로 배포합니다

나열된 단계를 수행하여 Trident 연산자를 수동으로 배포합니다.

1단계: Kubernetes 클러스터 검증

가장 먼저 해야 할 일은 Linux 호스트에 로그인하여 해당 호스트가 관리 중인 _ 작업 _ 을(를) 관리하고 있는지 확인하는 것입니다. "지원되는 Kubernetes 클러스터" 에 필요한 권한이 있어야 합니다.

> OpenShift에서는 뒤에 나오는 모든 예시에서 kubtl 대신 OC를 사용하고, OC login-u system:admin 또는 OC login-u kubbe-admin을 실행하여 먼저 * system:admin * 으로 로그인합니다.

Kubernetes 버전이 1.17 이후인지 확인하려면 다음 명령을 실행합니다.

```
kubectl version
```

Kubernetes 클러스터 관리자 권한이 있는지 확인하려면 다음 명령을 실행합니다.

```
kubectl auth can-i '*' '*' --all-namespaces
```

Docker Hub에서 이미지를 사용하는 Pod를 시작하고 Pod 네트워크를 통해 스토리지 시스템에 연결할 수 있는지 확인하려면 다음 명령을 실행합니다.

```
kubectl run -i --tty ping --image=busybox --restart=Never --rm -- \
  ping <management IP>
```

2단계: 운영자를 다운로드하고 설정합니다

> 21.01부터 Trident 연산자는 클러스터 범위입니다. Trident 연산자를 사용하여 Trident를 설치하려면 "트리엔오케스트레이터" CRD(사용자 정의 리소스 정의)를 생성하고 다른 리소스를 정의해야 합니다. Astra Trident를 설치하기 전에 다음 단계를 수행하여 운영자를 설정해야 합니다.

1. 의 최신 버전을 다운로드합니다 "Trident 설치 프로그램 번들" 다운로드 섹션에서 압축을 풉니다.

```
wget https://github.com/NetApp/trident/releases/download/v21.04/trident-
installer-21.04.tar.gz
tar -xf trident-installer-21.04.tar.gz
cd trident-installer
```

2. 적절한 CRD 매니페스트를 사용하여 '트리엔오케스트레이터' CRD를 생성합니다. 그런 다음 나중에 운영자가 설치를 인스턴스화하기 위해 '트리엔오케스트레이터' 사용자 정의 리소스를 만듭니다.

   다음 명령을 실행합니다.

```
kubectl create -f
deploy/crds/trident.netapp.io_tridentorchestrators_crd_post1.16.yaml
```

3. '트리엔오케스트레이터' CRD가 생성된 후 운용자 배치에 필요한 다음과 같은 자원을 생성한다.

   - 연산자를 위한 ServiceAccount입니다
   - ServiceAccount에 대한 ClusterRole 및 ClusterRoleBinding
   - 전용 PodSecurityPolicy
   - 작업자 자체

   Trident 설치 프로그램에는 이러한 리소스를 정의하는 매니페스트가 포함되어 있습니다. 기본적으로 연산자는 '삼원' 네임스페이스에 배포됩니다. 트리덴트 네임스페이스가 없으면 다음 매니페스트를 사용하여 만듭니다.

```
$ kubectl apply -f deploy/namespace.yaml
```

4. 기본 '삼중류' 네임스페이스 이외의 이름 공간에 운용자를 배치하려면 '세르이스계정.YAML', ' 클러스터구속력.YAML', 'operator.YAML'을 업데이트하고 'bundle.YAML'을 작성해야 합니다.

   다음 명령을 실행하여 YAML 매니페스트를 업데이트하고 '고객.YAML'을 사용하여 'bundle.YAML'을 생성합니다.

```
kubectl kustomize deploy/ > deploy/bundle.yaml
```

   다음 명령을 실행하여 리소스를 생성하고 연산자를 배포합니다.

```
kubectl create -f deploy/bundle.yaml
```

5. 배치한 후 작업자의 상태를 확인하려면 다음을 수행합니다.

```
$ kubectl get deployment -n <operator-namespace>
NAME                READY   UP-TO-DATE   AVAILABLE   AGE
trident-operator    1/1     1            1           3m

$ kubectl get pods -n <operator-namespace>
NAME                                READY   STATUS             RESTARTS
AGE
trident-operator-54cb664d-lnjxh     1/1     Running            0
3m
```

운영자 배포는 클러스터의 작업자 노드 중 하나에서 실행되고 있는 포드를 성공적으로 생성합니다.

> Kubernetes 클러스터에는 운영자의 인스턴스 * 하나가 있어야 합니다. Trident 연산자의 여러 배포를 생성하지 마십시오.

3단계: Trident를 설치한 후 트리젠터레이터 생성

이제 연산자를 사용하여 Astra Trident를 설치할 준비가 되었습니다! 이 작업을 수행하려면 '트리엔오케스트레이터'를 만들어야 합니다. Trident 설치 프로그램에는 'Trident Orchestrator'를 만들기 위한 예제 정의가 포함되어 있습니다. 이렇게 하면 '삼중덴트' 네임스페이스에서 설치가 시작됩니다.

```
$ kubectl create -f deploy/crds/tridentorchestrator_cr.yaml
tridentorchestrator.trident.netapp.io/trident created

$ kubectl describe torc trident
Name:        trident
Namespace:
Labels:      <none>
Annotations: <none>
API Version: trident.netapp.io/v1
Kind:        TridentOrchestrator
...
Spec:
  Debug:     true
  Namespace: trident
Status:
  Current Installation Params:
    IPv6:                      false
    Autosupport Hostname:
    Autosupport Image:         netapp/trident-autosupport:21.04
    Autosupport Proxy:
    Autosupport Serial Number:
    Debug:                     true
    Enable Node Prep:          false
    Image Pull Secrets:
    Image Registry:
    k8sTimeout:           30
    Kubelet Dir:          /var/lib/kubelet
    Log Format:           text
    Silence Autosupport:  false
    Trident Image:        netapp/trident:21.04.0
  Message:                  Trident installed  Namespace:
trident
  Status:                   Installed
  Version:                  v21.04.0
Events:
    Type Reason Age From Message ---- ------ ---- ---- -------Normal
    Installing 74s trident-operator.netapp.io Installing Trident Normal
    Installed 67s trident-operator.netapp.io Trident installed
```

Trident 연산자를 사용하면 ' Trident' 사양의 특성을 사용하여 Astra Trident가 설치되는 방식을 사용자 지정할 수 있습니다. 을 참조하십시오 "Trident 구축을 사용자 지정합니다".

'트리엔오케스트레이터'의 상태는 설치가 성공적이었음을 나타내며 설치된 Trident의 버전을 표시합니다.

상태	설명
설치 중	운영자는 이 '트리엔오케스트레이터' CR을 사용하여 Astra Trident를 설치하고 있습니다.
설치되어 있습니다	Astra Trident가 성공적으로 설치되었습니다.
제거 중	이 경우, 'pec.uninstall=true'로 인해 운용자가 Astra Trident를 설치 제거 중입니다.
제거되었습니다	Astra Trident가 제거되었습니다.
실패했습니다	운영자가 Astra Trident를 설치, 패치, 업데이트 또는 제거할 수 없습니다. 이 상태에서 자동으로 복구를 시도합니다. 이 상태가 지속되면 문제 해결이 필요합니다.
업데이트 중	운영자가 기존 설치를 업데이트하고 있습니다.
오류	트리젠터레이터(트리젠터레이터)는 사용하지 않는다. 다른 파일이 이미 있습니다.

설치 과정에서 트리엔오케스트레이터 상태가 설치 상태에서 설치 상태로 변경됩니다. 'Failed(실패)' 상태를 확인하고 운용자가 자체적으로 복구할 수 없는 경우 운용자의 로그를 확인해야 한다. 를 참조하십시오 "문제 해결" 섹션을 참조하십시오.

생성된 포드를 살펴보고 Astra Trident 설치가 완료되었는지 확인할 수 있습니다.

```
$ kubectl get pod -n trident
NAME                                READY   STATUS    RESTARTS   AGE
trident-csi-7d466bf5c7-v4cpw        5/5     Running   0           1m
trident-csi-mr6zc                   2/2     Running   0           1m
trident-csi-xrp7w                   2/2     Running   0           1m
trident-csi-zh2jt                   2/2     Running   0           1m
trident-operator-766f7b8658-ldzsv   1/1     Running   0           3m
```

또한 'tridentctl'을 사용하여 설치된 Astra Trident의 버전을 확인할 수도 있습니다.

```
$ ./tridentctl -n trident version
+----------------+----------------+
| SERVER VERSION | CLIENT VERSION |
+----------------+----------------+
| 21.04.0        | 21.04.0        |
+----------------+----------------+
```

이제 백엔드를 생성할 수 있습니다. 을 참조하십시오 "구축 후 작업".

배포 중 문제 해결에 대한 자세한 내용은 을 참조하십시오 "문제 해결" 섹션을 참조하십시오.

Trident 운영자 배포를 사용자 지정합니다

Trident 연산자를 사용하면 ' Trident' 사양의 특성을 사용하여 Astra Trident가 설치되는 방식을 사용자 지정할 수 있습니다.

속성 목록은 다음 표를 참조하십시오.

매개 변수	설명	기본값
'네임스페이스'	Astra Trident를 설치할 네임스페이스입니다	"기본값"
"버그"	Astra Trident에 대한 디버깅을 활성화합니다	거짓
'IPv6'	IPv6를 통해 Astra Trident를 설치합니다	거짓
k8sTimeout	Kubernetes 작업 시간이 초과되었습니다	30초
이제 자동 지원입니다	AutoSupport 번들을 NetApp에 자동으로 보내지 않습니다	거짓
"enableNodePrep"입니다	작업자 노드 종속성 자동 관리(* beta*)	거짓
자동 지원 이미지	AutoSupport 텔레메트리 컨테이너 이미지입니다	"NetApp/트리덴트 - 자동 지원: 21.04.0"
자동 지원 프록시	AutoSupport 텔레메트리 전송을 위한 프록시의 주소/포트입니다	"<a href="http://proxy.example.com:8888"" class="bare">http://proxy.example.com:8888"</a>
"제거"를 선택합니다	Astra Trident를 제거하는 데 사용되는 플래그입니다	거짓
로그포맷	사용할 Astra Trident 로깅 형식[text,json]	"텍스트"
트리덴티이미지	설치할 Astra Trident 이미지	"NetApp/트리덴트: 21.04"
'imageRegistry'입니다	'<registry FQDN>[:port][/subpath]' 형식의 내부 레지스트리 경로입니다	"k8s.gcr.io/sig-storage(k8s 1.17+) 또는 quay.io/k8scsi"
쿠벨레트디렉토리	호스트의 kubelet 디렉토리에 대한 경로입니다	"/var/lib/kubelet"
"위피"	Astra Trident를 완전히 제거하기 위해 삭제할 리소스 목록입니다	
'imagePullSecrets'	내부 레지스트리에서 이미지를 가져올 수 있는 비밀	
'controllerPluginNodeSelector'	Trident 컨트롤러 CSI 플러그인을 실행하는 Pod용 추가 노드 선택기. pod.spec.nodeSelector 과 동일한 형식을 따릅니다.	기본값 없음, 선택 사항

매개 변수	설명	기본값
'컨트롤러 PluginTolerations'	Trident 컨트롤러 CSI 플러그인을 실행하는 Pod의 허용 설정을 재정의합니다. pod.spec.Tolerations와 같은 형식을 따릅니다.	기본값 없음, 선택 사항
노드플러거노드 선택기	Trident Node CSI 플러그인을 실행하는 Pod용 추가 노드 선택기 pod.spec.nodeSelector 과 동일한 형식을 따릅니다.	기본값 없음, 선택 사항
노드 PluginTolerations	Trident Node CSI 플러그인을 실행하는 Pod의 허용 설정을 재정의합니다. pod.spec.Tolerations와 같은 형식을 따릅니다.	기본값 없음, 선택 사항

pec.namespace` 는 어느 네임스페이스 Astra Trident가 설치되어 있는지 나타내기 위해 트리엔터레이터에 지정됩니다. Astra Trident가 설치된 후에는 이 매개 변수 * 를 업데이트할 수 없습니다. 이렇게 하면 트리엔오케스트레이터 상태가 실패로 바뀝니다. Astra Trident는 네임스페이스 간에 마이그레이션할 수 없습니다.

자동 작업자 노드 준비는 비생산 환경에서만 사용할 수 있는 * 베타 기능입니다.

포드 매개 변수 포맷에 대한 자세한 내용은 을 참조하십시오 "노드에 Pod 할당".

'트리엔오케스트레이터'를 정의할 때 위에서 설명한 속성을 사용하여 설치를 사용자 정의할 수 있습니다. 예를 들면 다음과 같습니다.

```
$ cat deploy/crds/tridentorchestrator_cr_imagepullsecrets.yaml
apiVersion: trident.netapp.io/v1
kind: TridentOrchestrator
metadata:
  name: trident
spec:
  debug: true
  namespace: trident
  imagePullSecrets:
  - thisisasecret
```

다음은 노드 선택기를 사용하여 Trident를 구축하는 방법을 보여 주는 또 다른 예입니다.

```
apiVersion: trident.netapp.io/v1
kind: TridentOrchestrator
metadata:
  name: trident
spec:
  debug: true
  namespace: trident
  controllerPluginNodeSelector:
    nodetype: master
  nodePluginNodeSelector:
    storage: netapp
```

트리벤트오케스트레이터 인수를 통해 설치를 사용자 정의하려는 경우, 필요에 따라 수정할 수 있는 맞춤형 YAML 매니페스트를 생성하기 위해 'tridentctl'을 사용하는 것이 좋습니다.

Copyright Information

Copyright © 2022 NetApp, Inc. All rights reserved. Printed in the U.S. No part of this document covered by copyright may be reproduced in any form or by any means-graphic, electronic, or mechanical, including photocopying, recording, taping, or storage in an electronic retrieval system-without prior written permission of the copyright owner.

Software derived from copyrighted NetApp material is subject to the following license and disclaimer:

THIS SOFTWARE IS PROVIDED BY NETAPP “AS IS” AND WITHOUT ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE, WHICH ARE HEREBY DISCLAIMED. IN NO EVENT SHALL NETAPP BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.

NetApp reserves the right to change any products described herein at any time, and without notice. NetApp assumes no responsibility or liability arising from the use of products described herein, except as expressly agreed to in writing by NetApp. The use or purchase of this product does not convey a license under any patent rights, trademark rights, or any other intellectual property rights of NetApp.

The product described in this manual may be protected by one or more U.S. patents, foreign patents, or pending applications.

RESTRICTED RIGHTS LEGEND: Use, duplication, or disclosure by the government is subject to restrictions as set forth in subparagraph (c)(1)(ii) of the Rights in Technical Data and Computer Software clause at DFARS 252.277-7103 (October 1988) and FAR 52-227-19 (June 1987).

Trademark Information

NETAPP, the NETAPP logo, and the marks listed at http://www.netapp.com/TM are trademarks of NetApp, Inc. Other company and product names may be trademarks of their respective owners.